



# 07 — Temporal Context & Anchor Estimation

## Overview

Every encoded memory carries its temporal context as an intrinsic property of the trace, not as a separate metadata field. This is approximated via **multi-scale sinusoidal embeddings** (brain analogue: hippocampal time cells). At retrieval time, a temporal proximity bonus is added to each candidate's score.

Additionally, a **temporal anchor estimation** system (Stage 10) infers the query's intended time reference using embedded "temporal compass" probes — zero regex, zero extra LLM calls.

## Multi-Scale Temporal Embedding

### Encoding

For a Unix timestamp  $t$ , produce a  $d_t$ -dimensional vector:

$$\mathbf{t} = \bigoplus_{s \in \mathcal{S}} [\sin(\omega_s \cdot t), \cos(\omega_s \cdot t)]$$

Where  $\mathcal{S}$  is the set of time scales:

| Scale           | Period $T_s$ | Angular frequency $\omega_s = 2\pi/T_s$ |
|-----------------|--------------|-----------------------------------------|
| Minute          | 60 s         | $2\pi/60$                               |
| Hour            | 3,600 s      | $2\pi/3600$                             |
| Day             | 86,400 s     | $2\pi/86400$                            |
| Week            | 604,800 s    | $2\pi/604800$                           |
| Month (approx.) | 2,592,000 s  | $2\pi/2592000$                          |

| Scale  | Period $T_s$  | Angular frequency $\omega_s = 2\pi/T_s$ |
|--------|---------------|-----------------------------------------|
| Year   | 31,536,000 s  | $2\pi/31536000$                         |
| Decade | 315,360,000 s | $2\pi/315360000$                        |

Total temporal dimension:  $d_t = 2 \cdot |\mathcal{S}| = 14$

# Similarity

Two timestamps close on **any** scale have positive cosine similarity; separated by all scales have near-zero similarity:

$$\cos(\mathbf{t}_1, \mathbf{t}_2) = \langle \hat{\mathbf{t}}_1, \hat{\mathbf{t}}_2 \rangle$$

Vectors are L2-normalized before comparison.

# Temporal Anchor Estimation (Stage 10)

## Temporal Compass Probes

A fixed set of natural-language phrases with known time displacements:

| Probe Phrase                        | Displacement (seconds) |
|-------------------------------------|------------------------|
| "right now, today, at the moment"   | 0                      |
| "a few minutes ago, just now"       | −300                   |
| "an hour ago, earlier today"        | −3,600                 |
| "yesterday"                         | −86,400                |
| "a few days ago, earlier this week" | −259,200               |
| "last week"                         | −604,800               |
| "a couple weeks ago"                | −1,209,600             |
| "last month"                        | −2,592,000             |

| Probe Phrase                      | Displacement (seconds) |
|-----------------------------------|------------------------|
| "a few months ago"                | −7,776,000             |
| "last year, a year ago"           | −31,536,000            |
| "a few years ago"                 | −94,608,000            |
| "a long time ago, many years ago" | −315,360,000           |

## Algorithm

**Step 1: Embed probes.** Each probe phrase is encoded and L2-normalized at init time, producing a matrix  $\mathbf{P} \in \mathbb{R}^{n \times d}$ .

**Step 2: Compute similarities.** For query embedding  $\mathbf{q}$ :

$$\mathbf{s} = \mathbf{P} \cdot \mathbf{q} \in \mathbb{R}^n$$

**Step 3: Dead-zone gate.** If no past probe beats the "now" probe by at least `atemporal_margin` (default: `0.12`):

$$\max_{i>0} s_i - s_0 < \theta_{\text{atm}} \implies \text{return } t_{\text{now}}$$

**Step 4: Softmax with temperature.** Convert to weights:

$$w_i = \frac{\exp(s_i/T)}{\sum_j \exp(s_j/T)}, \quad T = \text{temperature} \quad (\text{default: } 0.15)$$

**Step 5: Weighted displacement.**

$$\Delta = \sum_{i=0}^{n-1} w_i \cdot d_i, \quad \text{anchor\_ts} = t_{\text{now}} + \text{round}(\Delta)$$

## Integration in Retrieval

When the query anchor differs from "now", the retrieval temporal embedding uses the **anchor timestamp** rather than current time:

$$\mathbf{t}_q = \text{temporal\_encode}(\text{anchor\_ts})$$

This biases retrieval toward episodes close to the inferred temporal reference point.

# Configuration

## TemporalProbe Constructor

| Parameter                              | Type     | Default    | Description           |
|----------------------------------------|----------|------------|-----------------------|
| encode_fn                              | Callable | (required) | Text encoder function |
| temperature $T$                        | float    | 0.15       | Softmax temperature   |
| atemporal_margin $\theta_{\text{atm}}$ | float    | 0.12       | Dead-zone margin      |

## RetrievalConfig (temporal parameters)

| Parameter                  | Type  | Default | Description                                  |
|----------------------------|-------|---------|----------------------------------------------|
| use_temporal               | bool  | True    | Enable temporal score component              |
| temporal_weight $\alpha_t$ | float | 0.25    | Weight of temporal similarity in final score |

# Key Invariants

1. Temporal embeddings are deterministic — same timestamp always produces same vector.
2. The temporal score is additive, not multiplicative — it nudges ranking, doesn't override semantic match.
3. The dead-zone gate prevents atemporal queries ("previous conversation") from triggering false temporal anchors.
4. Temperature  $T = 0.15$  produces peaked softmax — usually 1–2 dominant probes.
5. The temporal compass generalizes to any phrasing the encoder has seen (including multilingual expressions).